

# Documentación: Sistema de Predicción de Penales

Copa de Algoritmia y Programación UADE 2026 - Desafío 2

## 1. Descripción general

Este programa resuelve el desafío de predicción de penales. El objetivo es analizar el historial reciente de penales de un jugador rival y determinar hacia qué dirección pateó con mayor frecuencia. El resultado permite sugerir al arquero la dirección más probable del disparo.

Cada penal se representa con una letra: L para izquierda, R para derecha y C para centro. El programa lee la secuencia desde un archivo de texto, valida los datos, cuenta las direcciones y muestra la dirección dominante junto con su frecuencia.

## 2. Formato del archivo de entrada

El archivo debe llamarse penales.txt y debe estar en la misma carpeta que el archivo Python del programa. Debe contener una secuencia en una sola línea, formada solamente por las letras L, R y C.

```
LRRCLRRLLR
```

El programa usa readline(), por lo tanto toma la primera línea del archivo como secuencia de entrada.

## 3. Formato de salida

Al ejecutar la opción 2 del menú, el programa muestra un encabezado de resultado y luego imprime la salida principal en dos líneas: dirección dominante y cantidad de apariciones.

```
===== RESULTADO =====  
  
R  
5
```

## 4. Reglas de conteo y desempate

El programa cuenta cuántas veces aparece cada dirección en la secuencia. Luego compara los contadores para determinar cuál es la dirección más utilizada.

| Letra | Dirección | Contador   |
|-------|-----------|------------|
| L     | Izquierda | contador_l |
| R     | Derecha   | contador_r |
| C     | Centro    | contador_c |

En caso de empate, se aplica la prioridad táctica  $L > R > C$ . La función `direccion_dominante()` respeta esta regla iniciando la mejor dirección en L y cambiando solamente si otro contador es estrictamente mayor.

## 5. Estructura general del código

El código se divide en bloques y funciones para que cada parte tenga una responsabilidad clara. Esta organización mejora la legibilidad, evita código suelto y facilita las pruebas.

| Bloque | Función / parte       | Responsabilidad                                        |
|--------|-----------------------|--------------------------------------------------------|
| 1      | lee_archivo()         | Abrir el archivo y leer la primera línea.              |
| 1.1    | validar_secuencia()   | Validar vacío, longitud y caracteres permitidos.       |
| 2      | contar_direcciones()  | Contar apariciones de L, R y C.                        |
| 3      | direccion_dominante() | Determinar dirección más frecuente y resolver empates. |
| 4      | ejecutar_prediccion() | Coordinar lectura, validación, conteo y salida final.  |
| 5      | mostrar_menu()        | Mostrar menú interactivo y ejecutar opciones.          |

## 6. Funciones principales

### lee\_archivo(nombre\_archivo)

Abre el archivo en modo lectura, lee la primera línea con `readline()`, limpia el salto final con `strip()` y devuelve la secuencia junto con una bandera de error.

### validar\_secuencia(sequencia)

Controla que la secuencia no esté vacía, que tenga entre 1 y 1000 penales y que todos sus caracteres sean L, R o C en mayúscula.

### contar\_direcciones(sequencia)

Recorre la secuencia posición por posición y acumula la cantidad de tiros a izquierda, derecha y centro.

### direccion\_dominante(contador\_l, contador\_r, contador\_c)

Compara los contadores y devuelve la dirección dominante junto con su frecuencia. Respeta la prioridad de desempate  $L > R > C$ .

### ejecutar\_prediccion()

Ejecuta el flujo principal: leer archivo, validar secuencia, contar direcciones, calcular la dominante e imprimir el resultado.

### borrar\_pantalla()

Imprime varios saltos de línea para limpiar visualmente la consola y mejorar la presentación del menú.

### mostrar\_menu()

Muestra el menú principal, valida la opción ingresada y permite acceder al modo de uso, ejecutar la predicción, ver información del proyecto o salir.

## 7. Validaciones implementadas

Las validaciones buscan evitar que el programa siga procesando datos incorrectos. Si se detecta un error, se informa al usuario y no se realiza el conteo.

| Validación                   | Qué controla                                                                                          |
|------------------------------|-------------------------------------------------------------------------------------------------------|
| Archivo inexistente          | Se maneja con try/except FileNotFoundError en lee_archivo().                                          |
| Archivo vacío                | Si la secuencia leída es vacía, se muestra un error.                                                  |
| Longitud                     | La secuencia debe tener entre 1 y 1000 caracteres, incluyendo 1 y 1000.                               |
| Caracteres permitidos        | Solo se aceptan L, R y C en mayúscula.                                                                |
| Números, símbolos y espacios | Cualquier carácter distinto de L, R o C se considera inválido.                                        |
| Minúsculas                   | l, r y c se consideran inválidas porque la consigna permite L, R y C.                                 |
| Menú                         | try/except ValueError evita que el menú se rompa si el usuario ingresa letras o deja vacía la opción. |

## 8. Menú del programa

El programa incluye un menú interactivo para mejorar la experiencia del usuario. La evaluación es manual, por lo que el menú ayuda a presentar el sistema de forma clara.

```
===== MENU PRINCIPAL =====
1 - Modo de uso
2 - Ejecutar prediccion
3 - Acerca de
0 - Salir
```

La opción 1 explica cómo preparar el archivo. La opción 2 ejecuta la predicción. La opción 3 muestra información del proyecto y los integrantes. La opción 0 finaliza el programa.

## 9. Punto de inicio del programa

Al final del código se utiliza el siguiente bloque:

```
if __name__ == "__main__":
    mostrar_menu()
```

Este bloque indica que, si el archivo se ejecuta directamente, el programa debe iniciar mostrando el menú principal. Esto mejora la estructura del código y evita que el menú se ejecute automáticamente si el archivo fuera importado desde otro programa para probar funciones.

## 10. Casos de prueba recomendados

Estos casos permiten verificar que el programa funcione correctamente en situaciones normales, empates y entradas inválidas.

| Entrada en penales.txt | Resultado esperado |
|------------------------|--------------------|
| LRRCLRLLR              | R / 5              |
| LLLLL                  | L / 5              |

|               |                             |
|---------------|-----------------------------|
| CRCRCRR       | R / 4                       |
| LLRR          | L / 2                       |
| RRCC          | R / 2                       |
| CCCCC         | C / 5                       |
| LR2CL         | Error por carácter inválido |
| lrrc          | Error por minúsculas        |
| archivo vacío | Error por archivo vacío     |

## 11. Modo de uso final

- Crear un archivo llamado penales.txt.
- Colocar penales.txt en la misma carpeta que el archivo Python.
- Escribir una secuencia usando solamente L, R y C.
- Ejecutar el programa Python.
- Elegir la opción 2 - Ejecutar prediccion en el menú.
- Leer la dirección dominante y su cantidad de apariciones.

## 12. Integrantes

- Lucas Abad
- Thiago Albornoz
- Valentino Sarniguette
- Gaston Trezeguet
- Valentin Zaccari

Documento preparado para acompañar la entrega del Desafío 2. La documentación describe el comportamiento actual del código entregado y las validaciones implementadas.